

End-to-End Retail Data Engineering Pipeline

Project Report

Team Members:

Team Member 1 – Data Pipeline & Automation

Team Member 2 – Data Warehouse & Analytics

Technologies:

- Python, Pandas, PostgreSQL, Docker
- Apache Airflow, Power BI, GitHub

Date: September 2026

Table of Contents

- 1. Project Overview
- 2. Problem Statement
- 3. Project Objectives
- 4. Technology Stack
- 5. High-Level Architecture
- 6. Team Responsibilities
- 7. Data Warehouse Design (Star Schema)
- 8. ETL Pipeline Implementation
- 9. Apache Airflow Workflow Automation
- 10. Docker Configuration
- 11. Power BI Dashboard
- 12. Expected Deliverables
- 13. Learning Outcomes
- 14. Project Summary

1. Project Overview

This project aims to build an end-to-end Data Engineering pipeline using a retail sales dataset. The system will automate the process of extracting, transforming, storing, and visualizing retail transaction data using industry-relevant tools and practices. Instead of performing manual analysis in spreadsheets, the project will create a structured data warehouse and automated ETL workflow that can support business reporting and future analytical applications.

2. Problem Statement

Retail organizations generate large amounts of sales data from different products, customer segments, and geographical regions. Managing this data manually is inefficient and prone to errors. The objective of this project is to transform raw retail data into a clean, structured, and analytics-ready data warehouse through automated ETL processes and workflow orchestration.

3. Project Objectives

- Build an automated ETL pipeline for retail data
- Clean and transform raw sales data using Python
- Design a Data Warehouse using a Star Schema
- Store processed data in PostgreSQL
- Automate workflows using Apache Airflow
- Containerize services using Docker
- Create business dashboards using Power BI
- Maintain project versioning and documentation using GitHub

4. Technology Stack

Technology	Purpose
Python	Core programming language for ETL development
Pandas	Data manipulation and transformation
PostgreSQL	Relational database for data warehouse
Docker	Containerization of services

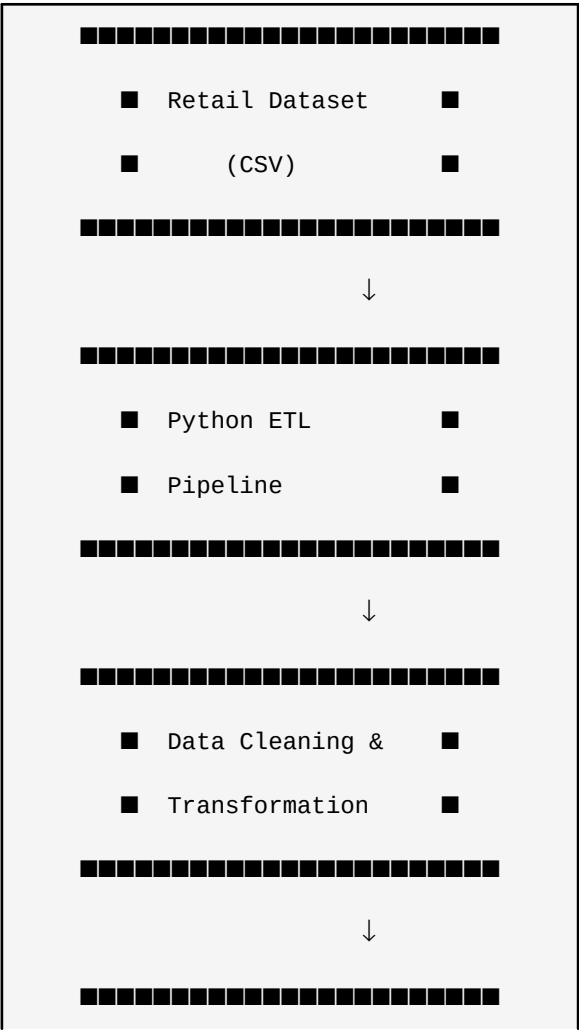
Apache Airflow	Workflow orchestration and automation
Power BI	Business intelligence and dashboard creation
GitHub	Version control and collaboration

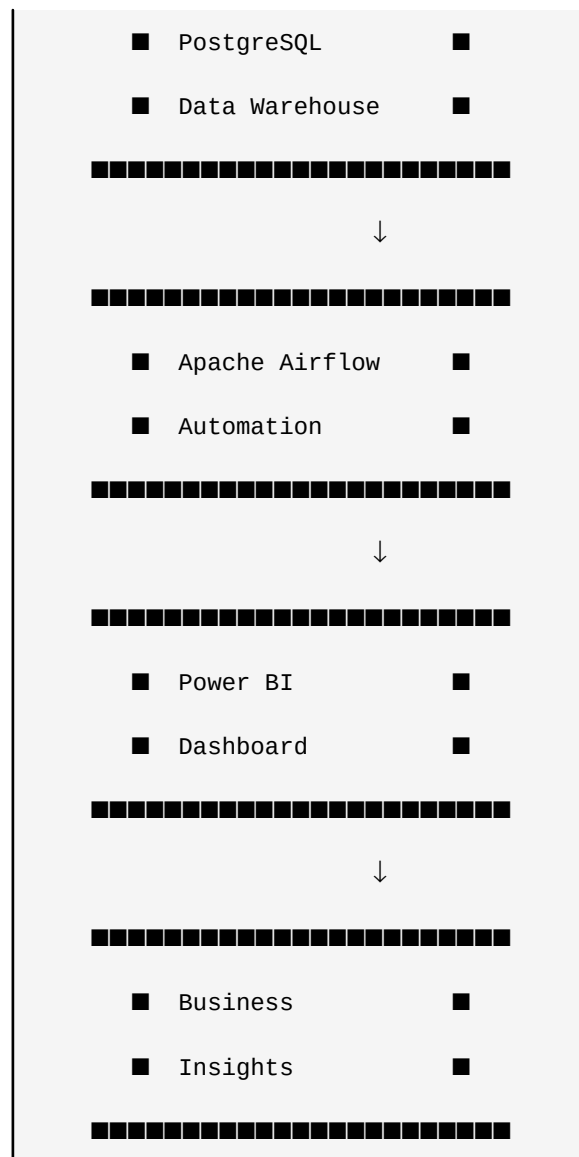
5. High-Level Architecture

The pipeline follows a sequential flow from raw data to business insights:

- Retail Dataset (CSV) → Python ETL Pipeline
- Data Cleaning & Transformation → PostgreSQL Data Warehouse
- Apache Airflow Automation → Power BI Dashboard
- Business Insights

Architecture Flow Diagram:





6. Team Responsibilities

Team Member 1 – Data Pipeline & Automation

Technologies: Python, Pandas, Apache Airflow, GitHub

Responsibilities:

- Data Ingestion
- Data Cleaning
- Data Transformation
- ETL Development
- Airflow DAG Creation
- Workflow Automation
- Documentation Support

Team Member 2 – Data Warehouse & Analytics

Technologies: PostgreSQL, SQL, Docker, Power BI

Responsibilities:

- PostgreSQL Setup
- Database Design
- Star Schema Implementation
- Fact & Dimension Tables
- Docker Configuration
- SQL Development
- Dashboard Creation

7. Data Warehouse Design (Star Schema)

A star schema is a dimensional modeling technique used in data warehousing. It consists of a central fact table surrounded by dimension tables, resembling a star. This design optimizes query performance for analytical workloads and simplifies business intelligence reporting.

Star Schema Components:

Fact Table: `fact_sales`

Column Name	Data Type	Description
sale_id	INTEGER (PK)	Unique identifier for each sale
date_id	INTEGER (FK)	Reference to date dimension
product_id	INTEGER (FK)	Reference to product dimension
customer_id	INTEGER (FK)	Reference to customer dimension
region_id	INTEGER (FK)	Reference to region dimension
quantity	INTEGER	Number of units sold
sales_amount	DECIMAL	Total sales value
profit	DECIMAL	Profit earned from sale

Dimension Tables:

Table Name	Key Columns	Purpose
date_dim	date_id (PK), date, year, month, quarter, day_of_week	Time-based analysis
product_dim	product_id (PK), product_name, category, sub_category, brand	Product hierarchy
customer_dim	customer_id (PK), customer_name, segment, gender, age_group	Customer segmentation
region_dim	region_id (PK), region_name, country, state, city	Geographical analysis

8. ETL Pipeline Implementation

The ETL (Extract, Transform, Load) pipeline is the core component that processes raw retail data and prepares it for analytics. The pipeline is implemented using Python and Pandas for efficient data manipulation.

ETL Process Steps:

- Extract: Load raw CSV data into Pandas DataFrame
- Transform: Clean data: handle missing values, remove duplicates, standardize formats
- Transform: Create calculated fields: profit margins, date components
- Transform: Validate data types and constraints
- Load: Insert cleaned data into PostgreSQL fact and dimension tables

Sample Python ETL Code Structure:

```
import pandas as pd
import psycopg2
from datetime import datetime

def extract_data(csv_path):
    """Extract data from CSV file"""
    df = pd.read_csv(csv_path)
    return df

def transform_data(df):
    """Clean and transform data"""
    # Handle missing values
    df = df.dropna()

    # Remove duplicates
    df = df.drop_duplicates()

    # Create date dimension
    df['date'] = pd.to_datetime(df['order_date'])
    df['year'] = df['date'].dt.year
    df['month'] = df['date'].dt.month

    # Calculate profit
    df['profit'] = df['sales'] - df['cost']

    return df
```

```
def load_data(df, db_config):
    """Load data into PostgreSQL"""
    conn = psycopg2.connect(**db_config)
    df.to_sql('fact_sales', conn, if_exists='append', index=False)
    conn.close()
```

9. Apache Airflow Workflow Automation

Apache Airflow orchestrates and automates the ETL pipeline through Directed Acyclic Graphs (DAGs). The DAG defines the workflow, task dependencies, and scheduling configuration.

Airflow DAG Structure:

- `extract_task`: PythonOperator to extract CSV data
- `transform_task`: PythonOperator to clean and transform data
- `load_task`: PythonOperator to load into PostgreSQL
- `validate_task`: PythonOperator to verify data quality

Sample DAG Configuration:

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from datetime import datetime, timedelta

default_args = {
    'owner': 'retail_team',
    'depends_on_past': False,
    'start_date': datetime(2024, 1, 1),
    'email_on_failure': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5)
}

dag = DAG(
    'retail_etl_pipeline',
    default_args=default_args,
```

```
description='Retail Data ETL Pipeline',
schedule_interval='@daily',
catchup=False
)

# Define tasks
extract = PythonOperator(
    task_id='extract_data',
    python_callable=extract_data,
    dag=dag
)

transform = PythonOperator(
    task_id='transform_data',
    python_callable=transform_data,
    dag=dag
)

load = PythonOperator(
    task_id='load_data',
    python_callable=load_data,
    dag=dag
)

# Set dependencies
extract >> transform >> load
```

10. Docker Configuration

Docker containerizes all services (PostgreSQL, Airflow, and application) to ensure consistent environments across development and deployment. This eliminates "works on my machine" issues and simplifies setup.

Docker Compose Services:

- postgres: PostgreSQL database container
- airflow-webserver: Airflow web interface
- airflow-scheduler: Airflow task scheduler
- airflow-worker: Airflow task executor
- etl-app: Custom Python ETL application

Sample docker-compose.yml:

```
version: '3.8'

services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: retail_warehouse
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secure_password
    ports:
      - '5432:5432'
    volumes:
      - postgres_data:/var/lib/postgresql/data

  airflow-webserver:
    image: apache/airflow:2.7.0
    environment:
      - AIRFLOW__CORE__EXECUTOR=LocalExecutor
      - AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=postgresql+psycopg2://admin:secure_password@postgres:5432/airflow
    ports:
      - '8080:8080'
```

```
depends_on:
- postgres

etl-app:
build: ./etl
depends_on:
- postgres
volumes:
- ./data:/app/data

volumes:
postgres_data:
```

11. Power BI Dashboard

Power BI connects to the PostgreSQL data warehouse to create interactive business intelligence dashboards. The dashboards provide real-time insights into sales performance, profitability, and regional trends.

Dashboard Pages:

- Sales Overview: Total revenue, units sold, profit margins
- Category Analysis: Sales by product category and subcategory
- Region Analysis: Geographic performance and regional comparisons
- Time Trends: Monthly, quarterly, and yearly sales patterns
- Customer Insights: Segmentation and customer behavior analysis

Key Performance Indicators (KPIs):

- Total Sales Revenue
- Total Profit
- Profit Margin (%)
- Units Sold
- Average Order Value
- Sales Growth Rate
- Top Performing Products

- Best Performing Regions

12. Expected Deliverables

Data Engineering Deliverables:

- Automated ETL Pipeline (Python scripts)
- Data Warehouse Design (PostgreSQL schema)
- Star Schema Implementation (fact and dimension tables)
- Airflow DAG (workflow automation)
- Dockerized Environment (docker-compose.yml)
- GitHub Repository (version control)

Analytics Deliverables:

- Power BI Dashboard (.pbix file)
- Business KPI Reports
- Category Analysis Visualizations
- Region Analysis Maps and Charts
- Profitability Analysis Reports

Documentation Deliverables:

- Project Report (this document)
- Architecture Diagram
- GitHub Repository with README
- Technical Documentation (setup guide, API docs)
- User Manual for Power BI Dashboard

13. Learning Outcomes

Through this project, team members will gain practical, hands-on experience in modern data engineering tools and practices used in industry.

- ETL Development: Building production-ready data pipelines
- Data Warehousing: Designing dimensional models with star schemas
- SQL & PostgreSQL: Advanced querying and database management

- Apache Airflow: Workflow orchestration and DAG creation
- Docker: Containerization and multi-service deployment
- Power BI: Business intelligence dashboard development
- GitHub Collaboration: Version control and team workflows
- End-to-End Data Engineering: Complete pipeline from source to insights

14. Project Summary

This project builds a complete retail data engineering solution that automates the flow of data from raw retail transactions to a structured PostgreSQL warehouse using Python, Docker, and Apache Airflow, while providing business-ready insights through Power BI dashboards.

One-Line Project Description:

Built an end-to-end Retail Data Engineering Pipeline using Python, PostgreSQL, Docker, Apache Airflow, and Power BI to automate ETL workflows, design a data warehouse, and deliver analytics-ready business insights.